

Interacción Hombre-Computadora

Implementación de la Interfaz Web

Primer Cuatrimestre 2026

Instituto Tecnológico de Buenos Aires

Grupo 6 - NUXU

Andrioli, Mateo (64042)

mandrioli@itba.edu.ar

Fraga, Santiago (64346)

safraga@itba.edu.ar

Giambelluca, Santino (64697)

sgiambelluca@itba.edu.ar

Goldbaum, Carolina (65112)

cgoldbaum@itba.edu.ar

Índice

1) Introducción	3
2) Requisitos Implementados	4
a) Requisitos Funcionales Obligatorios	4
i) Registrar cuenta	4
ii) Verificar cuenta	4
iii) Recuperar contraseña	4
iv) Cambiar contraseña	4
v) Iniciar sesión	4
vi) Cerrar sesión	4
vii) Gestionar, controlar y consultar dispositivos	4
viii) Gestionar, controlar y consultar rutinas	4
ix) Consultar acciones realizadas	4
x) Gestionar y consultar habitaciones	4
xi) Vincular dispositivos a habitaciones	5
xii) Gestionar y consultar hogares	5
b) Requisitos Funcionales Opcionales	5
i) Enviar notificaciones	5
ii) Restringir acceso a dispositivos, rutinas, habitaciones y hogares	5
iii) Consultar consumo eléctrico	5
iv) Planificar ejecución de rutinas	5
c) Requisitos No Funcionales Obligatorios	5
i) Tecnologías utilizadas	5
ii) Compatibilidad con navegadores	5
iii) Adaptación a resoluciones de pantalla	5
iv) Separación de estructura y presentación	5
v) Separación de estructura y comportamiento	5
d) Requisitos No Funcionales Opcionales	6
i) Validación de HTML	6
ii) Validación de CSS	6
iii) Diseño responsivo	6
iv) Accesibilidad (WAVE)	6
3) Decisiones de Usabilidad	7
4) Instrucciones de Instalación	8

1) Introducción

Este documento presenta el segundo entregable de la materia: la implementación del Sitio Web del proyecto NUXU. Es un servicio de gestión de dispositivos inteligentes para generar rutinas y gestionar habitaciones y hogares.

El grupo se enfocó en los conceptos teóricos, la documentación recomendada y los frameworks indicados para perfeccionar el diseño. Se tomaron como base los prototipos de la primera entrega, incorporando las mejoras sugeridas por la cátedra.

El resultado es un modelo fiel a los prototipos originales, tanto funcional como visualmente, que cumple los objetivos planteados para los modelos de usuario. Los cambios con respecto al prototipo se justifican más adelante.

2) Requisitos Implementados

a) Requisitos Funcionales Obligatorios

La implementación del Sitio Web arrancó con el *Dashboard* Principal y su barra de navegación horizontal, que enlaza a las páginas de “Home”, “Dispositivos”, “Rutinas”, “Historial”, “Configuración” y el *dropdown* de “Hogares”. Esta decisión de diseño está basada en el principio de “Flexibilidad y Eficacia en el Uso” de Jakob Nielsen. La barra de navegación le aporta al usuario una interacción rápida a través de estos *shortcuts*; tanto para usuarios experimentados e inexperimentados, ya que están acostumbrados a este tipo de componentes en su vida cotidiana. Inmediatamente después, se integró *Vue Router* para el ruteo de páginas y el flujo de interacción inicial.

i) Registrar y verificar cuenta

Como nuestro Sitio Web requiere de autenticación, se desarrollaron páginas para que el usuario pueda darse de alta y verificar su usuario en la plataforma. Para lograr este objetivo, se tuvo que tener en consideración la API provista por la cátedra para el manejo de datos del usuario.

En este caso en particular, se realizó un cambio sustancial que se consideró aporta más valor; se eliminaron las flechas de dirección del prototipo final -compárese **Figura 2** y **Figura 3**-. Éstas, en términos generales, resultaban molestas y se reemplazaron por la posibilidad de redirigirse a la página de “Inicio de Sesión”, estándar en las páginas más modernas. Además, el logo de la aplicación NUXU redirige a la página de “Bienvenida” -véase **Figura 1**-.



Figura 1: Página de Bienvenida

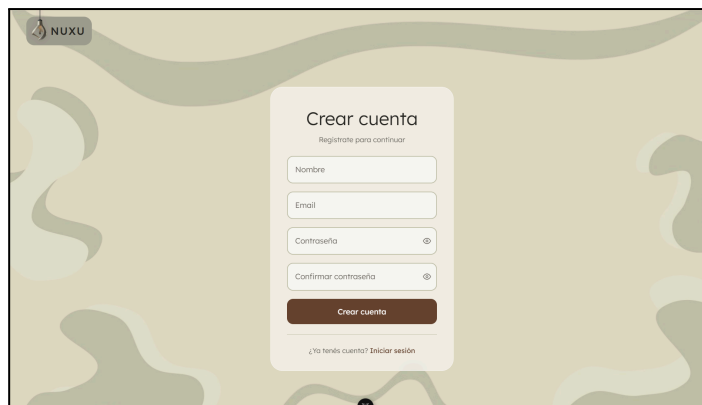


Figura 2: Página de Creación de Cuenta



Figura 3: Prototipo Final de la Página de Creación de Cuenta

ii) Recuperar contraseña

Si el usuario no recuerda su contraseña, el Sitio Web cuenta con la opción de “¿Olvidaste tu contraseña?” -véase **Figura 4-** en el panel de autenticación. Lo que lo hace fácilmente localizable es que está entre las credenciales y el botón de “Iniciar Sesión”, por lo que es muy probable que el usuario lo note a simple vista. El objetivo principal del diseño es reducir la carga cognitiva del usuario, haciendo trivial el proceso de recuperar su contraseña.



Figura 4: Ventana para Cambio de Contraseña

En caso de que el email ingresado sea válido, se da paso a la opción de ingresar un código único enviado a dicho email.

iii) Iniciar y cerrar sesión

Para el inicio y cierre de sesión se adoptó un diseño minimalista fiel al prototipo -compárese **Figura 5** con **Figura 6**-. Se mantuvo el fondo del prototipo con formas geométricas pero se centró la atención en el panel de autorización, sin sobrecargar al usuario con información irrelevante. Además, se eliminaron las tarjetas de autenticación SSO, ya que excede el alcance de la entrega y no está contemplado en el enunciado.

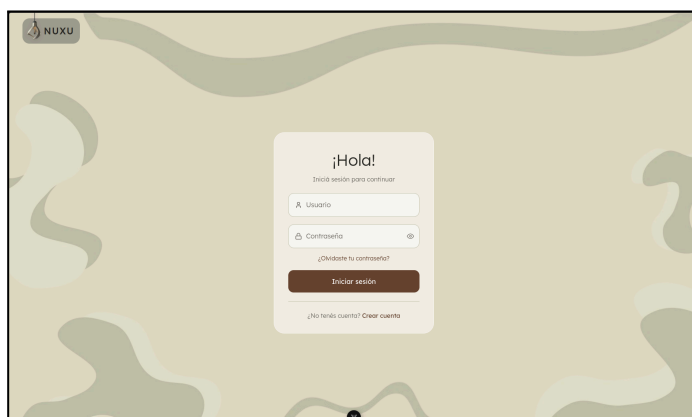


Figura 5: Página de Inicio de Sesión

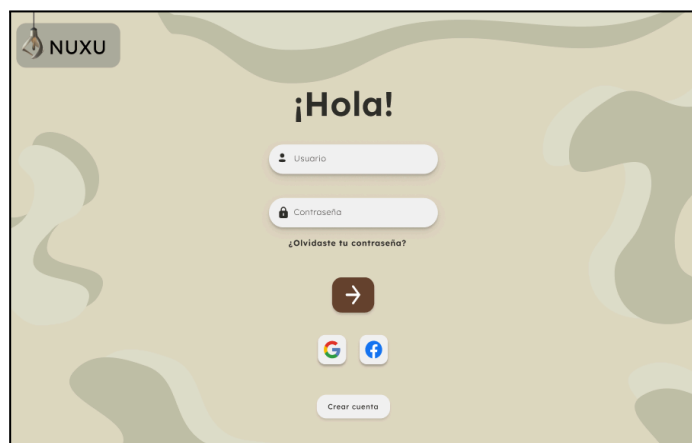


Figura 6: Prototipo final de la Página de Inicio de Sesión

Para cerrar sesión en la plataforma, se cuenta con la sección impresa en **Figura 7** -arriba a la derecha en el panel horizontal del *Dashboard*-. Al costado del perfil -donde se despliega un *dropdown* a la Configuración de Cuenta y Gestión de Hogares- se encuentra el botón accionable para cerrar la sesión del usuario.

Siguiendo el principio de “Prevención de Errores” de Jakob Nielsen, luego de presionar dicho botón se muestra un aviso de confirmación -véase **Figura 8**-. Esta política previene errores de alto costo al usuario y le brinda otra oportunidad de previsualizar su acción. Al confirmarse el aviso, se redirige instantáneamente al usuario a la página de “Inicio de Sesión”.

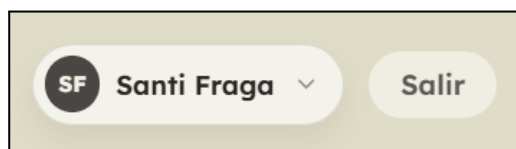


Figura 7: Botón para Cerrar Sesión

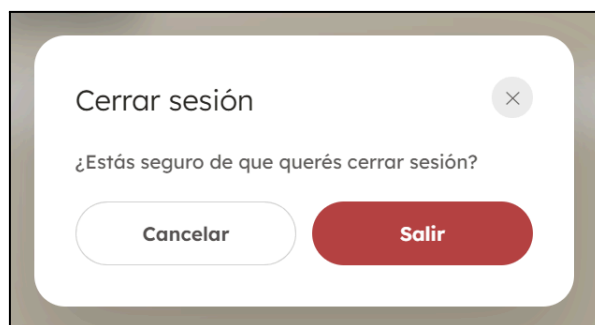


Figura 8: Aviso de Cerrar Sesión

iv) Gestionar, controlar y consultar dispositivos

Como se verá de aquí en adelante, todas las páginas tendrán su respectivo título que refuerza visualmente la opción elegida en la barra de navegación horizontal. Además, se evitó el uso de colores oscuros en superficies -compárese **Figura 9** con **Figura 10**-, ya que esto genera un mayor abarrotamiento y condiciona el peso visual de los elementos. Por adición, se hizo especial énfasis en que los elementos de la interfaz de usuario sean lo suficientemente contrastantes y rápidamente identificables. Estos cambios fueron implementados gracias a las correcciones recibidas acerca del prototipo final de la primera entrega.

Oportunamente, las vistas de todos los dispositivos se incluyen en el Anexo del documento.

Para agregar un dispositivo en el Sitio Web, se debe hacer *click* en la caja “Nuevo” -véase **Figura 9**-. Esta caja tiene un suave borde gris a su alrededor junto con un color distintivo también gris para indicar que su contenido es vacío -no es un dispositivo interactivo-. Se decidió posicionar el botón de agregado al comienzo de la cadena, ya que es un estándar en la industria y esto se condice con el modelo mental de nuestros modelos de usuarios.



Figura 9: Panel de Dispositivos del Sitio Web

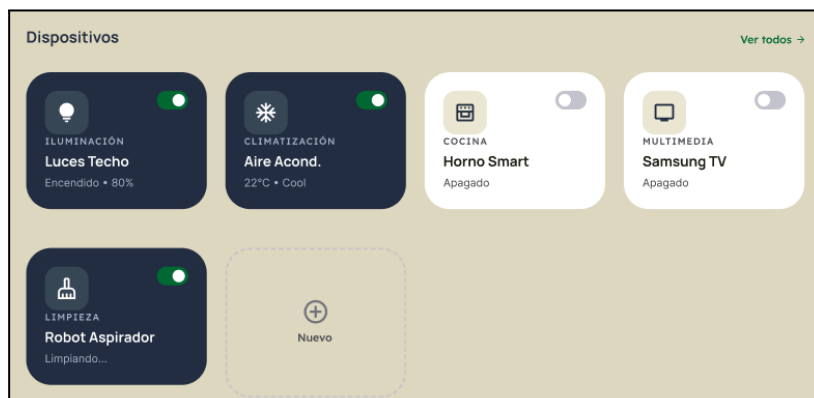


Figura 10: Panel de Dispositivos del Prototipo

En cuanto al comportamiento de los dispositivos, se realizó un cambio sustancial para algunos de estos. Se eliminó el *toggle* de encendido y apagado para los dispositivos de heladera, horno y puerta. Según la funcionalidad de dichos dispositivos, no es coherente que se puedan encender o apagar, sino modificar su comportamiento interno. Por ejemplo, es evidente la peligrosidad que tiene encender un horno de forma accidental. Este cambio de diseño está respaldado por el principio de “Prevención de Errores” de Jakob Nielsen, ya que de no haber modificado este comportamiento, un *slip* inconsciente podría derivar en una acción no deseada.

Contrario a la recomendación en la devolución de la primera entrega, no se implementó un botón flotante (FAB) para el agregado de nuevas entidades. Se consideró que el componente de botón flotante es más común encontrarlo en implementaciones Móvil, y al implementarlo generaría una hipotética fricción entre el usuario y el Sitio Web.

Al accionar el botón de agregado de dispositivos, se genera el panel que se muestra en **Figura 11**. En este, se puede ingresar el nombre del dispositivo, su tipo y el lugar dónde se lo quiere agregar. Al escoger el tipo, se le asigna al dispositivo automáticamente una imagen representativa. No se le permite al usuario que pueda modificar esta imagen ni el color de la caja del dispositivo, ya que así se mantiene una consistencia de colores y no afecta a la homogeneidad de la página. Esta política respeta el principio de “Consistencia y Estándares” de Jakob Nielsen.

Figura 11: Ventana de Creación de Dispositivo

Al momento de editar los dispositivos previamente agregados, se le dió mucha importancia en brindarle al usuario un estado claro que demuestre el riesgo de la siguiente acción. Para lograr esto, se agregó la siguiente leyenda en la parte inferior de la pantalla: “Modo edición activo: arrastrá los dispositivos para ordenarlos, o tocá uno para renombrarlo o eliminarlo”. Además, contornea todos los dispositivos de un color más oscuro e inhabilita el *toggle* de encendido/apagado y la caja de agregado de nuevo dispositivo -véase **Figura 12**-.

Siguiendo el principio de “Visibilidad del Estado del Sistema” de Jakob Nielsen, esta pista visual actúa como *feedback* para el usuario; le indica que la acción puede ser destructiva y que tenga especial cuidado al realizar cualquier modificación. De esta manera, se construye una confianza entre el Sitio Web y el usuario a través de una comunicación visual.



Figura 12: Modo Edición de Dispositivos

El panel para editar un dispositivo de forma individual se observa en **Figura 13**. Dentro de este mismo, se puede editar el nombre del dispositivo, cambiar la habitación en dónde se encuentra o eliminarlo permanentemente. Nuevamente, se persiste en el principio de “Visibilidad del Estado” para indicar que el botón “Eliminar” es riesgoso. En este caso, la caja contenedora es de color rojo para indicar que esta acción es destructiva.

Siguiendo el principio de “Prevención de Errores” de Jakob Nielsen, luego de presionar el botón “Eliminar” se muestra el aviso impreso en **Figura 14**. Este aviso previene al usuario de un eventual *slip* y le ayuda a recuperarse de este error. Por adición, fue una intención de diseño que el botón de “Eliminar” del Mensaje de Confirmación esté en el lado contrario que en el Panel de Edición -compárese **Figura 13** y **Figura 14**- para evitar un *double click* que destruya el dispositivo no intencionalmente.



Figura 13: Panel de Edición de un Dispositivo

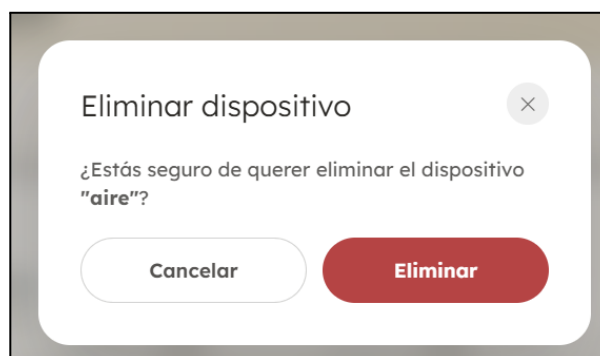


Figura 14: Mensaje de Confirmación

v) Gestionar, controlar y consultar rutinas

Para la creación y gestión de rutinas de dispositivos, se incorporó una página de rutinas exclusiva para esta funcionalidad -véase **Figura 15**-. En esta página se pueden añadir rutinas con el mismo flujo que con los dispositivos, *cliqueando* en la caja “Nuevo”.

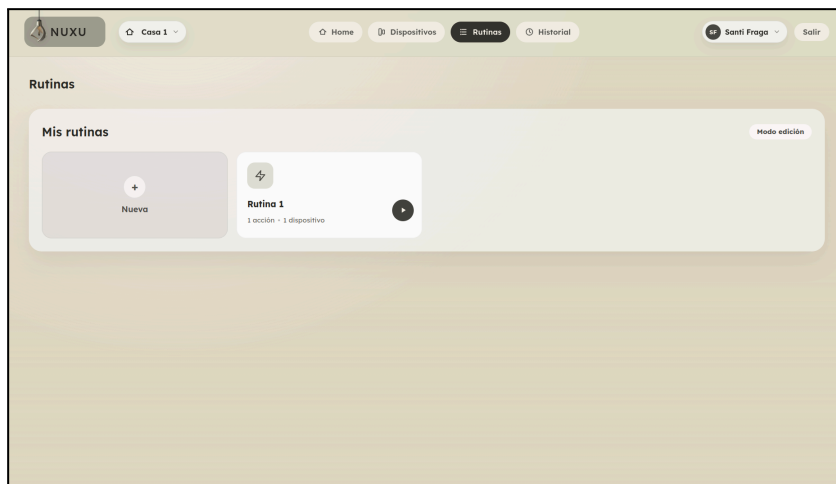


Figura 15: Página de Rutinas

Al accionar el botón de creación de rutina, se observa la **Figura 16**, se puede agregar el nombre del dispositivo y un ícono representativo. Al igual que con los dispositivos, no se permite que el usuario pueda modificar esta imagen, la de los dispositivos involucrados, ni el color de esta caja por la razón previamente esbozada. De igual forma, en esta misma ventana se pueden seleccionar todos los dispositivos existentes en el hogar -diferenciados por habitación- que vayan a formar parte de la rutina eventualmente.

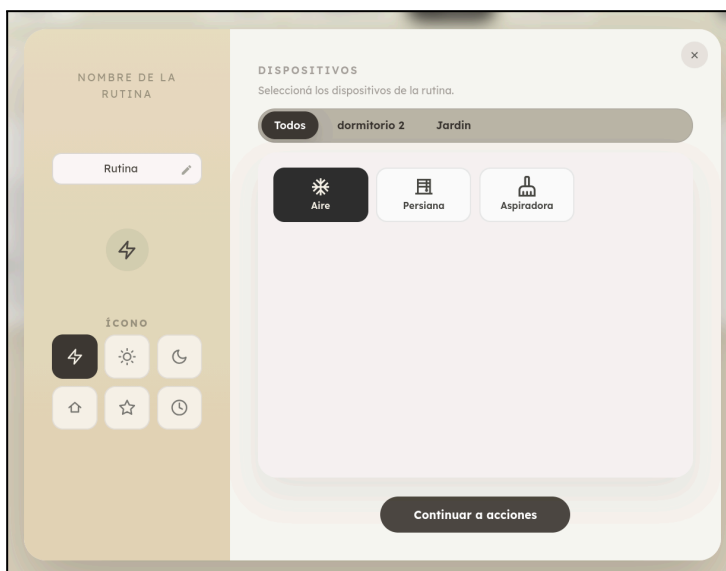


Figura 16: Ventana de Agregar Rutina

Según la recomendación de la cátedra, la creación de rutina fue implementada de manera que se vaya dando información de a pasos para evitar abrumar al usuario -véase **Figura 17**-. De igual modo, Mercedes -uno de nuestros modelos de personas- espera este comportamiento, ya que desearía

poder crear rutinas estructuradas y complejas desde su PC paso por paso; por ej. controlar la rutina de sueño de sus caniches blancos para evitar su mal humor por falta de sueño.

Cabe resaltar que nuestro segundo modelo de personas -Felipe- no será tenido en cuenta en esta entrega, ya que el principal uso que este prefiere darle a el proyecto es a través de su móvil, por lo que no le daría uso considerable al servicio alojado en el Sitio Web. Esto es debido a las diferencias que existen entre un Sitio Web y una aplicación Mobile. En el primero los componentes son mucho más robustos y complejos, por lo que no se condice con la practicidad y flexibilidad del uso que le daría Felipe al volver del trabajo o en el transporte público.

Para cada dispositivo, se pueden ejecutar únicamente acciones pertinentes al dispositivo en cuestión. De esta manera, no se generan inconsistencias dentro de la rutina; como bajar la temperatura de la puerta, o cambiar el color del horno, por ejemplo.

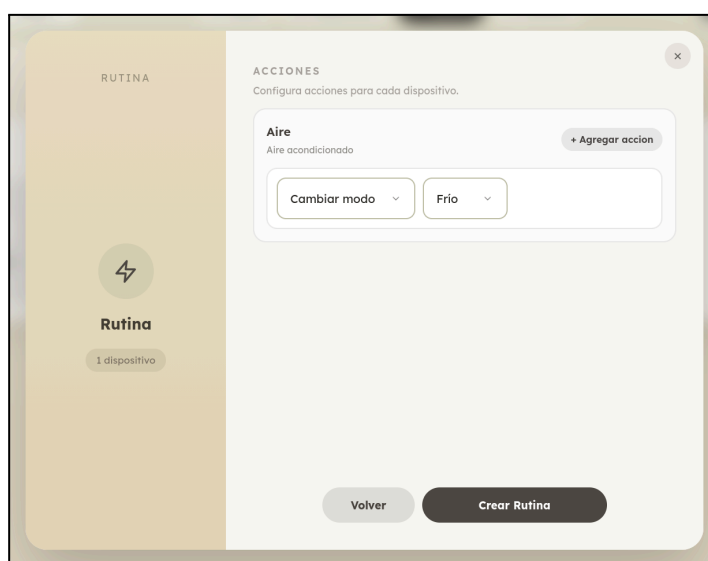


Figura 17: Edición de Rutina

Como el endpoint *GET/routines* de la API devuelve "List of all routines" del usuario autenticado, este no filtra por hogar. De esta manera, si por ejemplo un usuario es invitado a un hogar, este mismo no puede visualizar las rutinas de dicho hogar, sino que visualiza únicamente sus rutinas creadas anteriormente en otro hogar. Esta es una limitación de la API provista y no se consideró como un error.

vi) Consultar acciones realizadas

Para consultar las acciones realizadas en la página, hay un apartado de Historial donde se muestran en forma cronológica las operaciones realizadas -las más recientes primero- sobre todos los dispositivos de todas de todas las habitaciones del hogar -véase **Figura 18**-.



DISPOSITIVO	TIPO	ACCIÓN	FECHA	HORA	ESTADO
aire	Aspiradora	Iniciar	10/5/26	12:01:41 a. m.	OK
aire	Aspiradora	Pausar	10/5/26	12:01:40 a. m.	OK
aire	Aspiradora	Iniciar	10/5/26	12:01:40 a. m.	OK

Figura 18: Extracto de Página de Historial

Cada entrada del historial contiene la siguiente información:

- Nombre del dispositivo.
- Tipo del dispositivo.
- Información de la acción -apagado vs. encendido, cambio de modo, etc.-.
- Fecha y hora en la que se realizó la acción.

Se consideró que esta información es suficiente para que el usuario pueda identificar estos movimientos con facilidad y sin mayor carga cognitiva. De igual manera, en esta pestaña se puede filtrar por dispositivo, tipo de dispositivo, acción realizada y rango de fechas, evitándole al usuario una búsqueda excesiva entre las entradas del historial.

Como el Sitio Web para ordenador está orientado a las tareas más complejas -en contraposición al diseño móvil-, es idóneo que el apartado de Historial sea lo más detallado posible, aunque sin revelar información innecesaria para el usuario. El diseño del Historial está basado en el principio de “Reconocimiento en Lugar de Memoria” de Jakob Nielsen, ya que las entradas al estar listadas son fácilmente comparables entre sí.

vii) Gestionar y consultar habitaciones

Para el servicio que ofrece el Sitio Web, es de vital importancia poder tener múltiples habitaciones en cada hogar; como sucede en la vida cotidiana de los usuarios. Con respecto al prototipo final de la anterior entrega -véase **Figura 19**-, se cambió el enfoque de la creación de las habitaciones.



Figura 19: Prototipo final del Gestor de Habitaciones

Ahora en el Sitio Web, al momento de crear una nueva habitación se cuenta con un botón exclusivo para el agregado de una habitación tanto en la pantalla de home como en la de dispositivos -véase **Figura 20**-. Este cambio está sustentado en la prueba de usabilidad de la entrega anterior, ya que se mencionó en reiteradas ocasiones que el agregado de habitaciones puede resultar ambiguo o poco alcanzable.

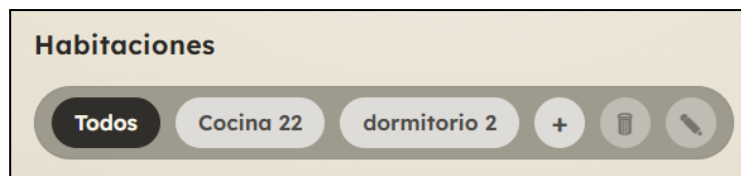


Figura 20: Gestor de Habitaciones

Al momento de agregar una habitación con el botón de agregado que se ve en la **Figura 20**, se despliega la ventana impresa en **Figura 21**. Por simplismo y consistencia con el Sitio Web, se decidió que este paso sea trivial y conciso, sin agregar pasos innecesarios como edición de color o ícono para la habitación.

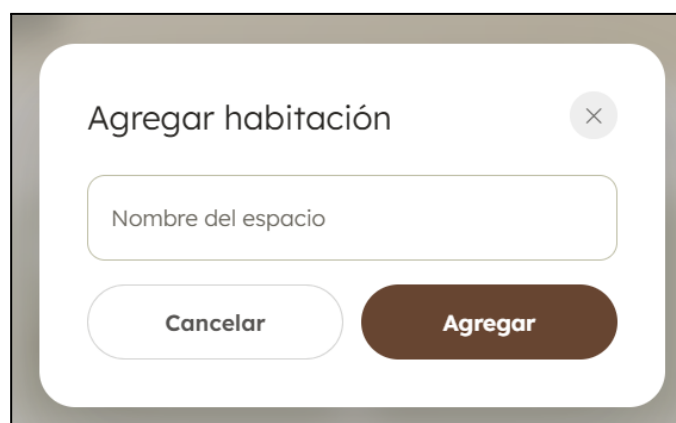


Figura 21: Ventana de Agregar Habitación

viii) Vincular dispositivos a habitaciones

Otra funcionalidad vital en nuestro Sitio Web es la vinculación de los dispositivos creados a habitaciones ya existentes. Se decidió que la selección de la habitación sea de tipo *dropdown* -véase **Figura 22**-, ya que es una manera rápida y accesible de ver todas las habitaciones disponibles. Esta ventana es accesible al estar en Modo Edición y *clickear* sobre un dispositivo para cambiar su nombre o habitación.



Figura 22: Ventana de Edición de Dispositivo

ix) Gestionar y consultar hogares

Otra funcionalidad clave para nuestra página es la posibilidad de que el usuario pueda gestionar múltiples hogares. Donde en cada uno de estos hogares existan diferentes habitaciones donde se pueden configurar dispositivos independientemente de cada habitación.

Esta sección se puede acceder desde el *dropdown* del Perfil -véase **Figura 23**-, ya que se consideró que la acción de gestionar hogares no es frecuentemente usada; por lo que no debería ser visible el 100% del tiempo.

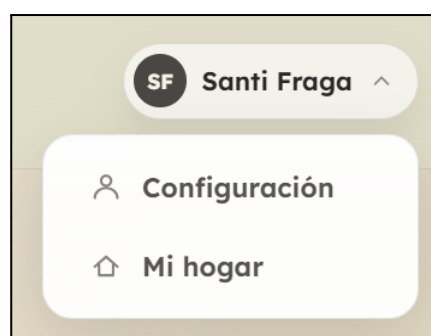


Figura 23: Dropdown para Configuración de Cuenta y Hogar

Dentro de la Configuración de Cuenta -véase **Figura 24**-, puede visualizarse la información del usuario, como su nombre o correo electrónico registrado. De la misma manera, puede cambiarse la contraseña de la cuenta desde el panel de Seguridad. Es intención de diseño que esta opción esté en un panel aparte ubicado en el lateral para evitar *slips* del usuario; fundamentado por el principio de “Prevención de Errores” de Jakob Nielsen.

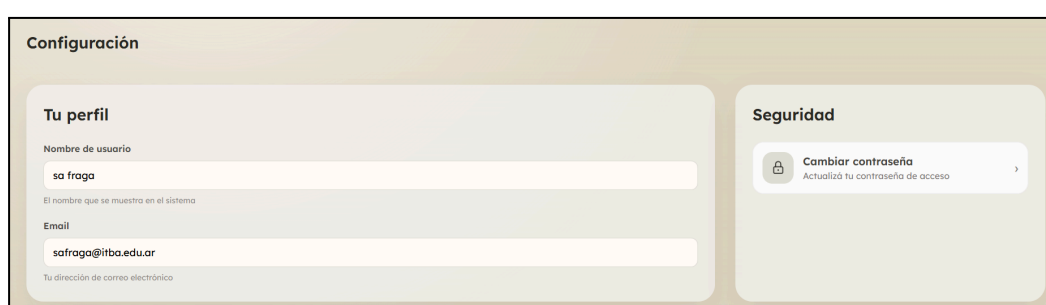


Figura 24: Configuración de Cuenta

Dentro de la Configuración de Hogar -véase **Figura 25**-, se tiene un *layout* parecido al *Dashboard* principal. Esta decisión de diseño está fuertemente basada en el principio de “Consistencia y Estándares” de Jakob Nielsen. Esto le aporta una familiaridad del producto y una consistencia interna al usuario del Sitio Web. Si no se hubiese pensado en esta política, la carga cognitiva del usuario lo forzaría a aprender el nuevo *layout*.

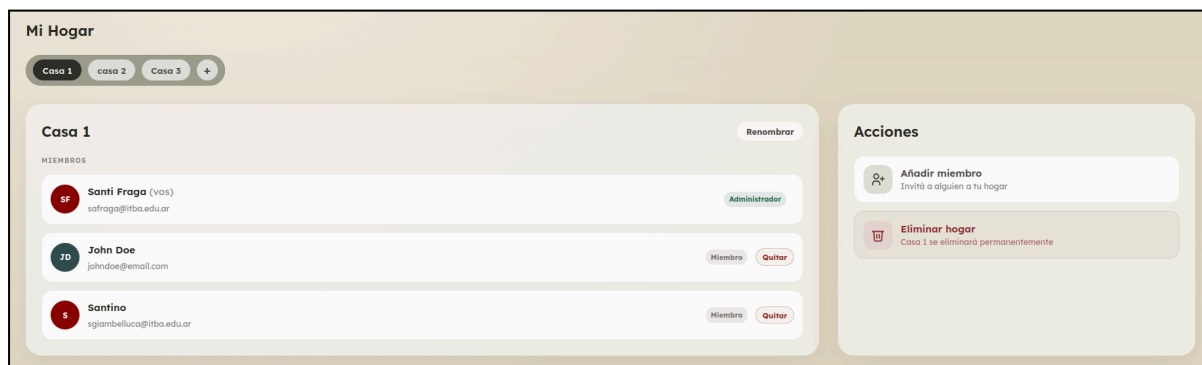


Figura 25: Configuración de Hogar

Dentro de esta página, se puede navegar entre los hogares de una manera análoga a como se haría con las habitaciones dentro del *Dashboard*, se puede consultar los miembros vinculados al hogar -y su rol-, renombrar, añadir/remover miembros nuevos al hogar, como también eliminar completamente el hogares -si se cuenta con permisos necesarios-.

Por otro lado, en la prueba de usabilidad realizada se discutió sobre la poca visibilidad de nuestro prototipo al momento de indicar en qué hogar se encuentra el usuario. Para mitigar esta problemática, se decidió implementar un *dropdown* -véase **Figura 26**- ubicado a la derecha en el panel horizontal de navegación donde se muestran todas los hogares vinculados al usuario.



Figura 26: Dropdown de Hogares

b) Requisitos Funcionales Opcionales

i) Rutinas programables

Se consideró que el tener rutinas programables aporta un valor agregado muy alto al Sitio Web. De esta manera, se aprovecha por completo la alta capacidad del Sitio Web para este tipo de tareas complejas. La ventana donde se programa la rutina está impresa en **Figura 27**.

Este requisito también se justifica con los requerimientos de Mercedes -una de nuestro modelo de personas-, ya que espera poder programar rutinas de cocina para automatizar el horno y la heladera, además de economizar su tiempo.

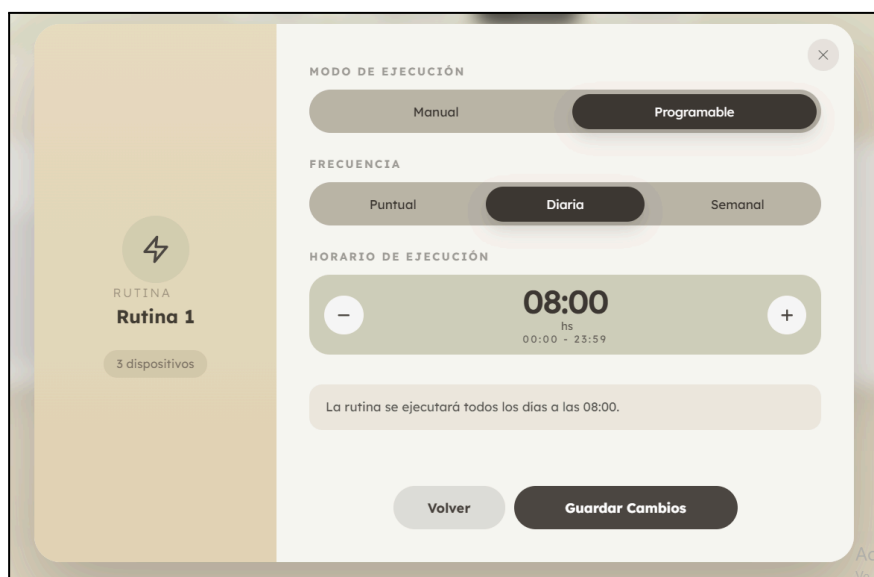


Figura 27: Ventana de Rutina Programable

c) Requisitos No Funcionales Obligatorios

El enunciado indica que a pesar de los Requisitos Funcionales Obligatorios, para la correcta ejecución de esta segunda entrega es necesario implementar y justificar los siguientes Requisitos No Funcionales Obligatorios. Estos requisitos aportan un gran valor en cuanto a la adaptabilidad, mantenibilidad y modularización del código.

i) Tecnologías utilizadas

Para la realización de la página se hizo uso del framework *Vue.js*, en conjunto con componentes de *Vue Router* y *Pinia*. Mediante dicho framework, el proyecto fue desarrollado utilizando JavaScript, HTML y CSS para garantizar interactividad, dinamismo y una arquitectura modular y escalable.

ii) Compatibilidad con navegadores

La página se probó tanto en *Mozilla Firefox* (en su versión 150.0.) como en *Google Chrome* (en su versión 147.0.7727.138). Se pudo verificar que los componentes se comportan de manera esperada y no hay conflictos en la visualización ni navegación de la página.

iii) Adaptación a resoluciones de pantalla

La página fue probada en resoluciones varias como 1280x720 px y 1920x1080 px, navegando distintas pantallas y realizando acciones pertinentes. En etapas tempranas, al achicar el ancho de la pantalla en la página de Inicio de Sesión, los *assets* .png del fondo se terminaban solapando entre sí cuando se corrían lo suficiente. Esto se generaba principalmente porque nuestros *assets* a pesar de ser .png, aún preservaban el fondo original del prototipo final extraído de *Figma*, por lo que se tuvo que actualizarlos y quitarles dicho fondo para aprovechar la transparencia del formato.

En general, se notó una buena adaptación a este tipo de resoluciones, respetando el comportamiento esperado de la página. Se verificó que todos los distintos componentes del Sitio Web mantengan su proporción y legibilidad. Esta confirmación le asegura al usuario una experiencia normalizada en monitores con distintas resoluciones.

iv) Separación de estructura y presentación

Con respecto a la correcta separación HTML/CSS, durante el desarrollo del código se usaron consistentemente propiedades *custom* para modularizar y mantener legible el código base.

Sin embargo, en *commits* iniciales, gracias a validaciones -explicadas más abajo- salieron a la luz ciertas declaraciones de estilo *inline*, lo que viola la regla de correcta separación de estructura/presentación. De esta manera, se tuvo que convertir estos *inlines* a clases CSS y se prestó atención a esto en declaraciones posteriores.

v) Separación de estructura y comportamiento

Con respecto a la correcta separación HTML/JavaScript, durante el desarrollo del código se ejecutó correctamente que toda la lógica compleja -como llamadas a la API, validaciones- estén en *stores* Pinia o en el bloque `<script setup>`.

Sin embargo, en el comienzo del proyecto ocurrió que el estado de algunas variables se cambiaban directamente en el template, como por ejemplo: `@click="activeRoomId = room.id"` ó `@click="showPass = !showPass"`. Para solucionar esta violación de separación, se reemplazaron por funciones que realicen la acción necesaria en `<script setup>`.

d) Requisitos No Funcionales Opcionales

i) Validación de HTML

Para garantizar la validación estándar HTML del W3C¹, se incorporó el linter `html-validate` junto con su plugin para *Vue* (`html-validate-vue`). Para ejecutar esta validación sobre todos los templates del proyecto, se agregó el script: `npm run validate:html`.

A mediados del proyecto, se realizó la primera ejecución de esta validación, el cuál detectó un total de 24 agrupados en tres categorías:

- **Estilos *inline*:** Gracias a la validación, se pudo notar que en reiteradas ocasiones se declaraba el estilo del componente de manera *inline*. Estos estilos estáticos se trasladaron a clases CSS en el bloque `<style scoped>` correspondiente. Por ejemplo, el gradiente que sigue al cursor se refactorizó para actualizar su posición mediante referencias de template y manipulación desde el script, eliminando el `:style` del template.
- **Botones sin tipo explícito:** Luego de la validación, se modificaron algunos botones dispersos que carecían del atributo `type`. Esto conlleva un riesgo de envíos accidentales de formulario. Para solucionarlo, se agregó `type="button"` en cada caso individualmente.
- **Input sin tipo explícito:** Análogo al punto anterior. Se recalcó que el campo de edición del nombre del hogar en `SettingsPage.vue` no declaraba `type="text"` de manera explícita.

ii) Validación de CSS

De la misma manera que ocurrió con HTML, para garantizar el estándar CSS del W3C se incorporó el linter `stylelint` junto con el preset `stylelint-config-standard` y el parser `postcss-html`, que permite analizar los bloques `<style scoped>` dentro de los componentes de *Vue*. Para ejecutar el validador, se agregó el script `npm run validate:css`.

A mediados del proyecto, se realizó la primera ejecución de esta validación, la cuál detectó errores. Se corrigieron los errores listados a continuación:

- **Valor inválido de propiedad CSS:** En la página `AuthLayout.vue`, se encontró la declaración `object-fit: left`, que es sintácticamente incorrecta ya que `object-fit` no acepta `left` como valor. Esta declaración se reemplazó por `object-fit: contain`, cuyo valor es el adecuado para contener una imagen dentro de sus dimensiones sin deformarla en el proceso.
- **Selectores duplicados:** En las páginas `AcControls.vue` y `OvenControls.vue` se encontraron reglas CSS con el mismo selector definidas dos veces consecutivamente con las propiedades repartidas entre ambas declaraciones. Para arreglar este error, se unificaron en una única regla por selector.

¹ El Consorcio World Wide Web (W3C) es una organización internacional sin ánimo de lucro de interés público donde las organizaciones miembro, un equipo de personal a tiempo completo y el público colaboran para desarrollar estándares web.

iii) Diseño responsivo

En cuanto a un diseño responsivo, el Sitio Web está diseñado para funcionar correctamente en el rango de resoluciones de escritorio comprendido entre 1280 px y 1920 px de ancho. Dentro de ese rango, la interfaz se adapta mediante las siguientes técnicas:

1. **Layouts Flexibles:** Todas las páginas utilizan *CSS Flexbox* y *CSS Grid* como sistema de empaquetado. Los contenedores principales no tienen anchos fijos, sino que tienen anchos máximos combinados con unidades relativas. Esto permite que el contenido se distribuya fluidamente al cambiar el tamaño de la ventana.
2. **Media Queries:** Los componentes que requieren ajustes puntuales en determinados anchos utilizan *Media Queries* para reorganizar su estructura. *AppLayout.vue* adapta su grilla de navegación, *LogsPage.vue* ajusta su padding y los controles de dispositivos modifican su disposición interna en *viewports* más ajustados.
3. **clamp():** Los títulos principales de las páginas de autenticación y Landing utilizan la función *clamp()* para escalar el tamaño de fuente de forma fluida entre un valor mínimo y máximo. Gracias a esto se eliminan grandes saltos al redimensionar la ventana del Sitio Web.
4. **Unidades relativas para espaciado/tamaño:** El espaciado interno y el tamaño de elementos visuales están expresados en unidades relativas. Esto garantiza que las proporciones se mantengan en distintas resoluciones dentro del rango objetivo.
5. **Posicionamiento sin coordenadas fijas:** Los elementos decorativos y de layout utilizan posicionamiento por flujo normal, *Flexbox* o *Grid* en lugar de coordenadas absolutas fijas. Gracias a esto se evitan desbordamientos o superposiciones al variar el *viewport*.

iv) WAVE

Para garantizar la superación del análisis del validador *WAVE* sin errores, se realizaron correcciones en cuatro áreas principales del Sitio Web.

1. Se corrigieron los problemas de contraste de color, reemplazando los valores de texto secundario que no alcanzaban la relación mínima de 4.5:1 exigida por *WCAG AA* por un color sólido que sí la cumple sobre fondos blancos.
2. Se agregó un enlace de salto de navegación ("Saltar al contenido principal") visible al recibir el foco con teclado, permitiendo a usuarios de teclado y lectores de pantalla omitir la barra de navegación.
3. Se añadió soporte de la tecla *Escape* en todos los modales de la aplicación, eliminando una barrera de navegación para usuarios que no utilizan *mouse*, en cumplimiento del criterio *WCAG 2.1 SC 2.1.1*.
4. Se implementó la actualización dinámica del título de página por ruta, mejorando la orientación para usuarios de lectores de pantalla que navegan entre secciones.

3) Decisiones de Usabilidad

Las decisiones de usabilidad se fundamentan en la teoría de clase -basándonos principalmente en las heurísticas de Jakob Nielsen-, los modelos de personas y las correcciones del prototipo anterior. A continuación se detallan las decisiones clave de esta etapa.

a) Diseño

El diseño prioriza el minimalismo para evitar el abarrotamiento y limitar la información visible simultáneamente. Esta política está estrechamente ligada con el principio de “Diseño Estético y Minimalista” de Jakob Nielsen. De esta manera, la interfaz no cuenta con elementos meramente ornamentales; está compuesta únicamente de componentes funcionales y que logran los objetivos principales del usuario. Este reduccionismo evita distracciones del usuario, y aumenta su productividad al momento de navegar por el Sitio Web.

En cuanto a la paleta de colores, se respetó la misma paleta que el prototipo final -véase **Figura 28**-. Se consideró que esta paleta es de una gama cálida y serena, fundamentado sobre la prueba de usabilidad realizada.

Color	HEX
	#f4f4f4
	#dfddc8
	#cfcdb4
	#c0c0a8
	#9e9b8e
	#2596be
	#413e37
	#292824

Figura 28: Paleta de colores de la página

Impulsado por el estilo minimalista, se decidió incorporar una tipografía liviana, sin ornamentos ni trazas agresivas para dar una sensación de confort y familiaridad. La tipografía más usada en el Sitio Web es Lexend. Esta decisión fue tomada debido a su gran legibilidad en pantalla. Esto, en consecuencia, reduce la fatiga visual en interfaces densas de datos. Además, como la fuente está disponible en Google Fonts, no requiere de una licencia adicional para el Sitio Web.

Durante la implementación del Sitio Web, se aseguró que el lenguaje utilizado sea el cotidiano para nuestros modelos de personas. No se pueden encontrar términos ni frases sofisticadas o muy técnicas del *scope*, sino que al contrario. Palabras como “Home”, “Dispositivos”, “Rutinas”, o “Historial” son las que esperan nuestros modelos de personas al encontrarse una aplicación de este estilo -no hay ambigüedad en los términos usados-.

De igual manera, se invirtió atención en que los íconos sean lo más representativos posibles, tanto para los dispositivos como las distintas secciones del Sitio Web. Por ejemplo, al comparar la vista de hogares del prototipo final -véase **Figura 29**- y el Sitio Web -véase **Figura 26**-, es clara la distinción entre las vistas. En el prototipo final, el ícono de Cambio de Hogar le resultó ambiguo a ciertos usuarios en las pruebas de usabilidad de la entrega anterior. Ahora -en la implementación final- hay una pista visual muy clara, ya que el Cambio de Hogar tiene un ícono de casa, seguido del nombre de la casa lo cuál es mucho más significativo que el diseño anterior.

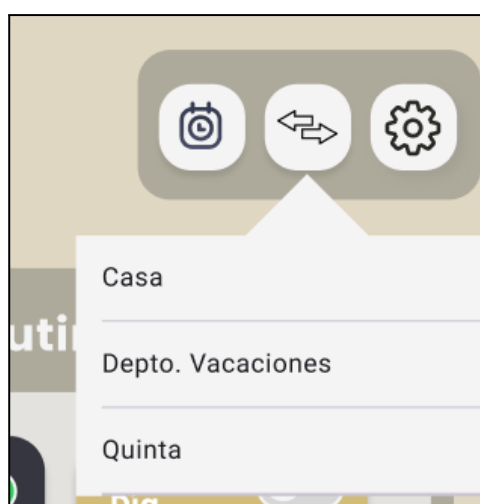


Figura 29: Dropdown de Hogares del Prototipo Final

Estos puntos se basan en el principio de “Coherencia entre el Sistema y el Mundo Real” de Jakob Nielsen.

Siguiendo el principio de “Reconocimiento en Lugar de Memoria” de Jakob Nielsen, el diseño de los dispositivos se pensó de manera que el usuario reconozca fácilmente el estado interno del dispositivo sin tener que acceder a este mismo necesariamente. Por ejemplo, desde el *Dashboard* principal de dispositivos, se puede comparar a simple vista si una cortina está cerrada -con el label “Cerrado”-, si un parlante está pausado -con el label “Detenido”-, etc. Análogamente, el estado de las rutinas son igualmente reconocibles a simple vista. De esta manera los estados son rápidamente comparables, sin necesidad de recordar altas cantidades de información simultáneamente - véase **Figura 9** - .

Otra justificación de este diseño es que Mercedes -uno de nuestros modelos de personas- desearía tener imperativamente una confirmación visual del estado de los dispositivos mientras cocina, como por ej. la temperatura precisa del horno y la heladera.

Según el principio de “Consistencia y Estándares” de Jakob Nielsen, es importante que nuestro Sitio Web persista en todo momento sobre un mismo idioma; en este caso, el español. Por este motivo se decidió traducir al castellano cualquier modismo angloparlante existente en nuestro Sitio Web.

b) Navegación

En cuanto a la navegación de la página, tuvimos especial preocupación por que el usuario sepa en todo momento dónde está parado. Por este motivo, respetamos nuestro prototipo final al incluir una barra de navegación con el título que corresponde a dicha página. De esta manera, el diseño se apoya en el principio de “Visibilidad del Estado del Sistema” de Jakob Nielsen.

De igual manera, el diseño del Sitio Web incorpora el principio de “Controles y Libertad del Usuario” de Jakob Nielsen, ya que todas las cajas de dispositivos, rutinas, avisos de confirmación, etc., tienen una cruz arriba a la derecha o un botón de “Cancelar” que minimiza o cancela *-undo-* la acción en cuestión. Esta política crea en el usuario una sensación de libertad y confianza con el Sitio Web, ya que los errores no forzados no son penalizados.

c) Botones y campos

En relación a los botones *clickables* de la página, todos tienen sombras, elevación u fondo que los distingue de su entorno. Por ejemplo, al pasar el ratón sobre las cajas de los dispositivos, estos se “elevan” y muestran una pequeña sombra difuminada para indicar su selección.

De la misma manera, los campos donde se puede introducir texto tienen un formato estándar acompañado por una leyenda. Estas consideraciones se basan fuertemente en los modelos mentales de nuestros potenciales usuarios.

Durante el *testing* del Sitio Web, incorporamos un capping en los campos de textos de 25 caracteres, lo cuál es suficientemente extenso para la gran mayoría de casos. En caso de que el texto completo no entre en el marco de la caja, se muestra con tres puntos suspensivos al final del nombre.

d) Jerarquía y composición

Durante la implementación del diseño en la primera entrega, se hizo especial énfasis en lograr una jerarquía definida para los distintos elementos dentro de las páginas, además una clara separación de estas mismas.

Para lograr estos objetivos, en todas las páginas existe una separación vertical de colores entre la zona del header -barra de navegación, configuración, selección de hogar- y la zona de interacción con los dispositivos y rutinas.

Para la fácil identificación de los componentes -por ej. “Dispositivos” vs. “Habitaciones”-, los elementos se agrupan en cajas bordeadas con un fondo más claro -respetando la paleta de colores- para dar la sensación de que son componentes distintos.

e) Mensajes

Para los mensajes de errores, se aseguró de que las notificaciones emergentes sean de color rojo para que sean rápidamente perceptibles desde la perspectiva del usuario -véase **Figura 30**-. Además, las acciones críticas -ej. eliminar un hogar- se agruparon en “zonas de peligro” para aportar un grado más de atención -véase **Figura 31**-.

En contraposición, las acciones de creación o guardado como “Añadir Miembro” -**Figura 31**- y los mensajes de confirmación -véase **Figura 32**- son de color verde para indicarle al usuario que son acciones constructivas, y no hay penalización si se comete algún error.

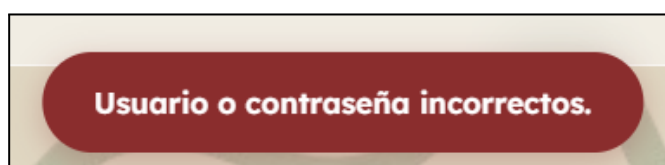


Figura 30: Mensaje de Error al Iniciar Sesión



Figura 31: Sección Eliminar Hogar



Figura 32: Mensaje de Confirmación al Cambiar la Configuración del Dispositivo

La elección de los mensajes y los colores de estos está fundamentada también en Mercedes -una de nuestros modelos de personas-, la cuál desearía tener una retroalimentación visual muy clara y lógica -con colores verdes/rojos- del estado de sus acciones en dispositivos y rutinas.

4) Instrucciones de Instalación

Para realizar la correcta y completa instalación de la aplicación, se debe contar previamente con los siguientes requisitos. Estos requisitos listan las versiones mínimas de los componentes de los que depende la ejecución de la página y de las versiones mínimas de los navegadores soportados.

Componente	Versión mínima
<i>Node.js</i>	Versión 20.19 o superior
<i>npm</i>	Incluido con <i>Node.js</i>
<i>Git</i>	Cualquier versión reciente funciona

Navegador	Versión mínima
Google Chrome	140
Microsoft Edge	140
Mozilla Firefox	140

Además, obviamente se debe contar con conexión a Internet -para la instalación de dependencias- y un editor de código.

A continuación, se listan los pasos de instalación necesarios para poder compilar y ejecutar el Sitio Web de manera local o levantarlo en un servidor y verificar su posterior instalación.

1. Clonar el repositorio

```
$: git clone https://github.com/matsdah/HCI_TP2
$: cd HCI_TP2
```

2. Instalar dependencias. Desde el directorio frontend/:

```
$: cd frontend
$: npm install
```

3. Configurar variables de entorno. Este archivo no se incluye en el repositorio por razones de seguridad. Crear el archivo frontend/.env.local con el siguiente contenido:

```
VITE_API_BASE_URL=https://hci.it.itba.edu.ar/api
VITE_API_KEY=<clave-de-grupo>
VITE_WS_URL=https://hci.it.itba.edu.ar/
VITE_EMAILJS_SERVICE_ID=service_glk7w6o
VITE_EMAILJS_TEMPLATE_ID=template_qr0g0rd
```


VITE_EMAILJS_PUBLIC_KEY=mZuzJXVh4OV-CaBwx

4. Servidor de desarrollo. Inicia en `http://localhost:5173`

```
$: cd frontend/
$: npm run dev
```

5. Compilar para producción. Este comando ejecuta la verificación de tipos y genera los archivos estáticos optimizados en el directorio `frontend/dist/`.

```
$: npm run build
```

6. Desplegar en un servidor. El resultado de la compilación es una aplicación de página única. Los archivos de `frontend/dist/` deben servirse desde un servidor HTTP estático. Dado que la navegación utiliza rutas del historial del navegador (*createWebHistory*), el servidor debe redirigir todas las rutas a *index.html*.

a. Vista local: Levanta un servidor local apuntando a `dist/` en el puerto 4173.

```
$: npm run preview
```

b. Nginx: Servidor de producción.

```
server {
    listen 80;
    root /var/www/nuxu/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

7. Verificación de la instalación: Una vez desplegado, acceder desde un navegador compatible a la *URL* del servidor. El sitio debe mostrar la pantalla de bienvenida de NUXU. Navegar a la pantalla de Inicio de Sesión e iniciar sesión con una cuenta registrada para confirmar la conectividad con la API de la cátedra.

8. Cambiar la URL de la API. Como se indica en las recomendaciones para el instructivo de instalación, a continuación se muestra una forma de cambiar la URL para otra API distinta.

En *frontend/.env.local* antes de correr el *build*:

VITE_API_BASE_URL=https://ejemplo.com/api

5) Conclusión

La realización de esta segunda entrega fue todo un logro para el grupo. En la mayoría de nuestros casos, esta entrega fue el primer acercamiento real que tuvimos al desarrollo Web. Convertir el diseño de las distintas páginas de *Figma* a código significó un desafío y nos llevó mucho tiempo en equipo; principalmente porque debimos aprender un lenguaje y una filosofía de programación completamente nueva para nosotros. De igual manera, al haber tenido modificaciones en la API provista a pocos días del *deadline*, tuvimos que refactorizar el código para que siga funcionando correctamente.

Para llevar a cabo este trabajo tuvimos que dividir roles dentro de la implementación del código y ponernos de acuerdo que partes de código editar para lograr las features necesarias.

Una lección valiosa que nos llevamos es la importancia de desarrollar un buen diseño de prototipo desde el principio. Nos ocurrió algunas veces que, al no haber implementado una página específica en el prototipo, tuvimos que iterar nuevamente para poder dilucidar un diseño de modelo que cumpla con las expectativas. Sin embargo, nuestro prototipo fue lo suficientemente extenso y detallado para tener una base sólida en la gran mayoría de páginas.

Por otro lado, durante el desarrollo del Sitio Web absorbimos mucha información valiosa de las funcionalidades de *Vue*, *Vue Router* y *Pinia*, y pudimos implementar buenas prácticas vistas en las clases teóricas.

6) Anexo

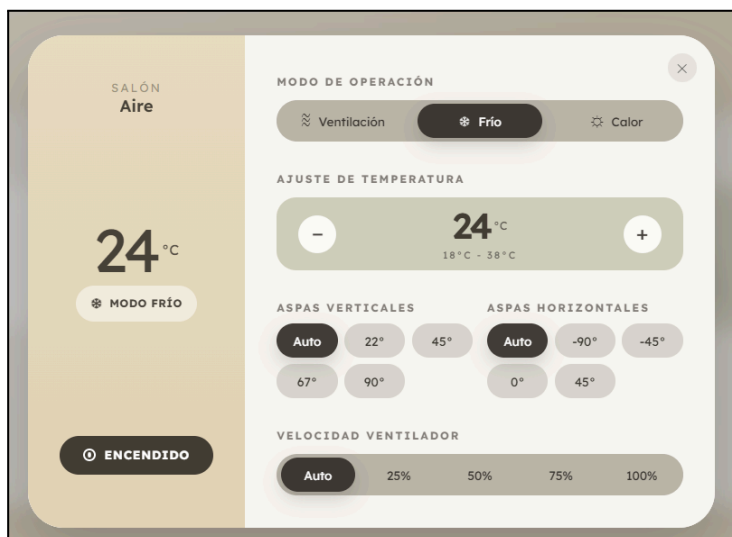


Figura 33: Vista de Aire Acondicionado

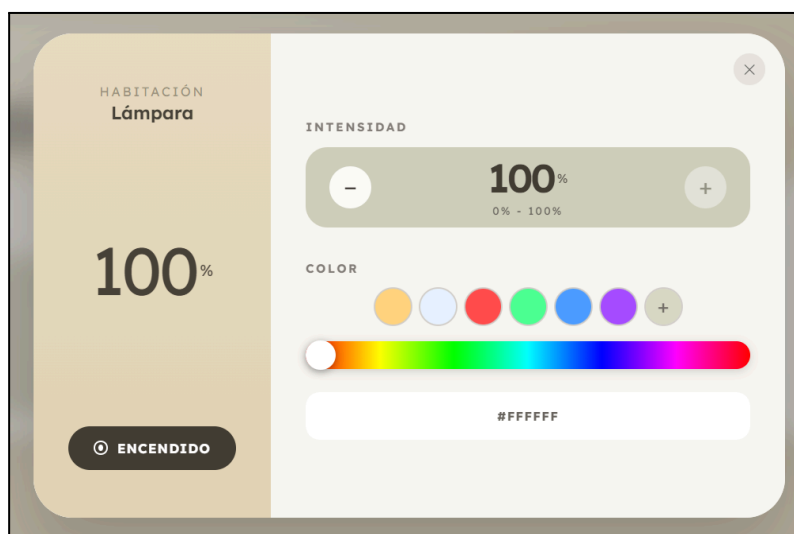


Figura 34: Vista de Lámpara



Figura 35: Vista de Alarma



Figura 36: Vista de Persiana



Figura 37: Vista de Puerta

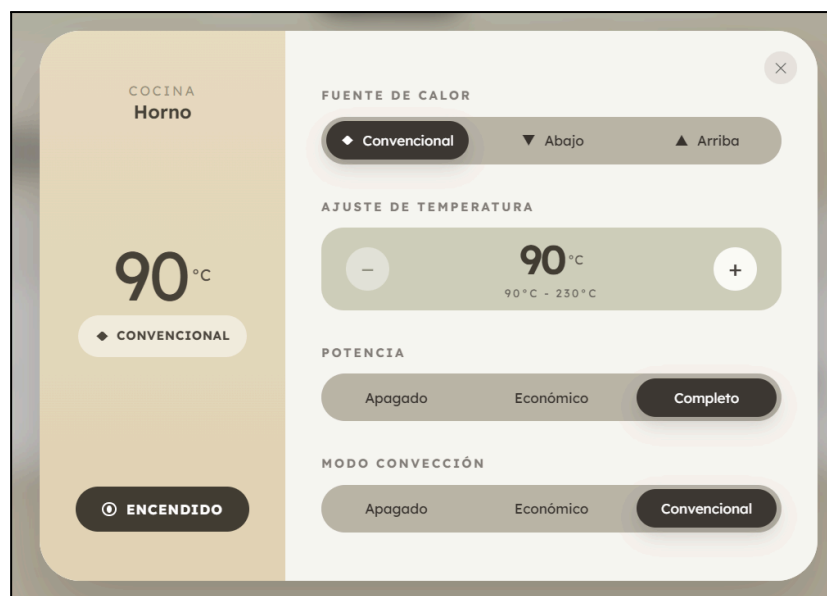


Figura 38: Vista de Horno



Figura 38: Vista de Heladera

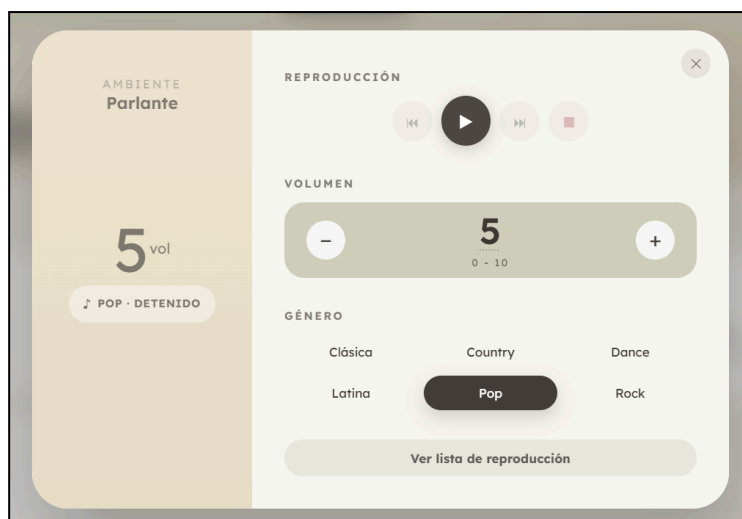


Figura 40: Vista de Parlante

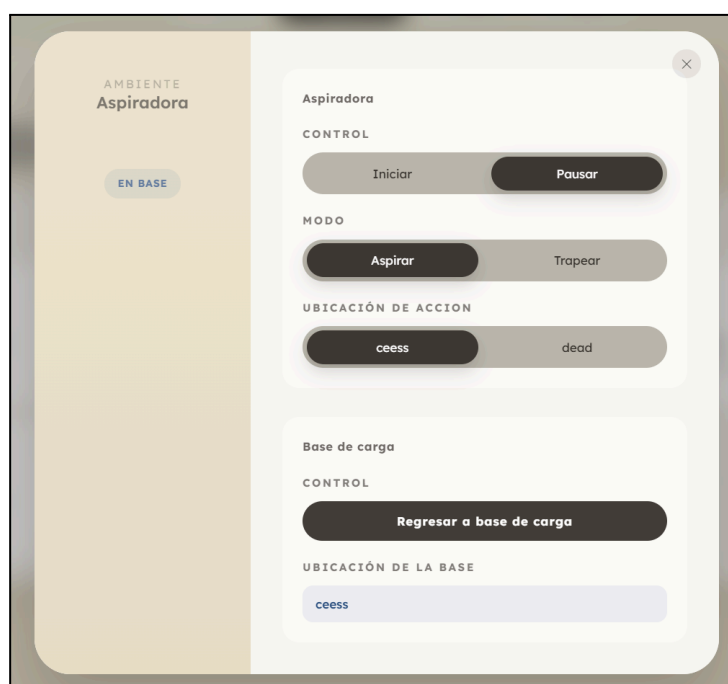


Figura 41: Vista de Aspiradora